Durable Records: One Presence and Survival Law for the Local Stores of a Rings Node

Ryan J. Kung

Abstract

A Rings node keeps security state (transaction replay windows), privacy state (onion entry guards) and custody state (relay mail) in local key-value stores on two platform families: a native file system and a browser's IndexedDB. Five open defects of these stores reduce to two predicates: presence, whether a record that should exist exists, and survival, whether a completed write outlives a crash. We give one model for both. A store's authority is a point of the product of a crash chain $\mathsf{Vol} \sqsubset \mathsf{Cch} \sqsubset \mathsf{Dur}$ and a meaning chain $\mathsf{Reb} \sqsubset \mathsf{Ret} \sqsubset \mathsf{Aut}$; each point fixes what an absent record means and which obligations the platform must discharge. We choose a per-record presence marker as the witness that separates a lost authoritative record from a never-written one, prove it sound and complete for partial loss, and prove that no witness in the same failure domain detects total loss. We state the crash obligation per platform, verified against the Chromium, Gecko and WebKit sources (only strict durability flushes in all three), forbid eviction for authoritative records, and give a custody law that makes relay mail survive a restart between admission and consumption, with the impossibility result that bounds what any receiver can do for mail that has fallen behind live traffic.

I. Problem

The defects, each named by its issue:

#915
: An absent native replay record is indistinguishable from a stream never seen, so its stream restarts from First; on non-unix targets a completed rename can roll back.

#923
: The browser replay store evicts its least recently used row at its row cap, so a retained stream can restart from First.

#912
: Browser replay writes use the default durability hint, so a completed admission can roll back after an OS crash.

#911
: The entry-guard store is disposable: a crash can lose or tear the guard set, re-selecting the first hop.

#936
: Relay mail admitted into the replay window and queued, but not taken before a restart, is refused as a replay on redelivery and lost.

#915, #923 are presence defects; #912, #911 are survival defects; #936 is a presence defect of a fact that lives in no store at all (that the queued mail was never consumed). Section II fixes the model, Section III the authority lattice, Sections IV–VIII the five laws, and Section IX the order of implementation.

Ryan J. Kung can be reached at ryankung@ieee.org. Manuscript created October 23, 2026. This document specifies the durability model of the local stores of Rings before 1.0 (rings-core, modules storage, message::replay and swarm::inbox; rings-node, module onion::entry_guard). It is the law for issues #911, #912, #915, #923 and #936 of RingsNetwork/rings.

II. Model

Definition 1 (Store). A store is a finite partial map $\sigma : \mathcal{N} \rightharpoonup \mathcal{V}$ from record names to values. Its operations are $\mathrm{put}(n, v)$, $\mathrm{rm}(n)$ and $\mathrm{get}(n)$, each returning Ok or Err; scan returns one classification per name (Section IV). For FileStorage the name of key k is $\mathrm{name}(k) = \mathrm{hex}(\mathrm{SHA1}(k))$; for IdbStorage it is k.

Definition 2 (History, completion). A history is $h = o_1 \cdots o_k \, o_{k+1}^{?}$: operations $o_1..o_k$ returned, o_{k+1} may be in flight. Let $\sigma_i = o_i \circ \cdots \circ o_1(\sigma_0)$, with $\sigma_{k+1} = \sigma_k$ when nothing is in flight. An operation is completed iff it returned Ok.

Definition 3 (Crash, recovery image). A crash ends the process (process crash) or the operating system (system crash: kernel panic, power loss). The recovery image $R_P(h)$ is the set of maps the next open of the store can read on platform P after a crash at the end of h.

Definition 4 (Loss events). A loss event is a change of the store's physical state that no operation of the store made:

- $\lambda_{\mathrm{part}}(S)$, partial loss: the records of $S \subsetneq \mathrm{dom}\,\sigma$ vanish (media error repaired by fsck, an operator removing or renaming one file, a completed rename rolled back by a platform that does not discharge Obligation 2);
- λ_{tot}, total loss: the whole failure domain vanishes (the store's directory, an IndexedDB database or origin, a process heap).

Loss events are what the platform or the operator does, not what an adversary forges: a party that can write the store can also write any witness of it, so nothing below

authenticates a store, and nothing models a state the program and platform cannot reach.

A. Obligations

Obligation 1 (Atomicity, A). $\forall \sigma' \in R_P(h).\ \forall n.\ \sigma'(n) \in \{\sigma_i(n) : i \le k+1\} \cup \{\bot\}$: a record is never torn.

Obligation 2 (Survival, D). $\forall \sigma' \in R_P(h).\ \forall n.\ \sigma'(n) \in \{\sigma_k(n), \sigma_{k+1}(n)\}$: every completed write and removal survives a process or system crash, and the one in flight is either whole or absent. D implies A.

Obligation 3 (No eviction, N). $\mathrm{put}(n, v)$ on σ yields $\sigma[n \mapsto v]$ or returns $\mathsf{Err}(\mathsf{BudgetExhausted})$ with σ unchanged; no operation removes a name it was not given.

Obligation 4 (Witnessed presence, W). After any history and any partial loss $\lambda_{\mathrm{part}}(S)$, scan reports every $n \in S \cap \mathrm{dom}\,\sigma_k$ as $\mathsf{Missing}(n)$, and reports $\mathsf{Missing}(n)$ only for an n whose record some put made durable.

Obligation 5 (Fail closed, F). A record that does not decode is reported and kept; only its owner or an operator removes it.

III. The authority lattice

Definition 5 (Crash chain). $C = \mathsf{Vol} \sqsubset \mathsf{Cch} \sqsubset \mathsf{Dur}$ with obligations $O(\mathsf{Vol}) = \emptyset$ ($R_P(h) = \{\emptyset\}$ at every restart), $O(\mathsf{Cch}) = \emptyset$ (any $\sigma_i(n)$, $\bot$ or a torn record), $O(\mathsf{Dur}) = \{A, D\}$.

Definition 6 (Meaning chain). $M = \mathsf{Reb} \sqsubset \mathsf{Ret} \sqsubset \mathsf{Aut}$, read as what $\sigma'(n) = \bot$ means to the owner:

- Reb: $\bot$ is the owner's default; any record may be dropped;
- Ret: $\bot$ is "never written, or retired by the owner"; a loss is a liveness failure;
- Aut: $\bot$ is "never written"; a loss is a safety failure.

$O(\mathsf{Reb}) = \emptyset$, $O(\mathsf{Ret}) = \{N\}$, $O(\mathsf{Aut}) = \{N, W, F\}$.

Definition 7 (Authority). An authority is $a = (c, m) \in C \times M$, ordered componentwise, with $O(c, m) = O(c) \cup O(m)$. $C \times M$ is a lattice (a product of chains, meet and join componentwise) and O is monotone: $a \sqsubseteq b \Rightarrow O(a) \subseteq O(b)$.

Law 1 (Admissibility). Let $\mathrm{req}(\mathsf{Reb}) = \mathsf{Vol}$ and $\mathrm{req}(\mathsf{Ret}) = \mathrm{req}(\mathsf{Aut}) = \mathsf{Dur}$. A store may be opened at (c, m) only if $c \sqsupseteq \mathrm{req}(m)$.

Justification. Under $c \sqsubset \mathsf{Dur}$ a crash may lose a completed write, so $\bot$ after recovery can be a loss; a meaning above Reb asserts that $\bot$ is not a loss. □

The existing RecordAuthority is the two-point chain $\mathsf{Disposable} = (\mathsf{Cch}, \mathsf{Reb}) \sqsubset \mathsf{Authoritative} = (\mathsf{Dur}, \mathsf{Aut})$ (#909, #910, #914). The law adds two points: $\mathsf{DurableCache} = (\mathsf{Dur}, \mathsf{Reb})$ for state whose loss is a downgrade, and $\mathsf{Retained} = (\mathsf{Dur}, \mathsf{Ret})$ for custody. MemStorage is $(\mathsf{Vol}, -)$: admissible only at Reb.

Table I is the whole classification. The two rows marked with new issues are inadmissible points the law exposes outside the five issues: relay carriers are the only copy of held mail (#1021), and a provider given no replay store runs replay at $(\mathsf{Vol}, \mathsf{Aut})$ (#1022).

Table I
Stores of a node and the authority the law assigns

Store	Today	Law
DHT data (native, browser)	(Cch, Reb)	(Cch, Reb)
relay carriers in it	(Cch, Reb)	(Dur, Ret) #1021
Measure, evidence	(Cch, Reb)	(Cch, Reb)
Entry guards	(Cch, Reb)	(Dur, Reb) #911
Replay (native)	(Dur, Aut)*	(Dur, Aut) #915
Replay (browser)	(Cch, Reb)†	(Dur, Aut) #912,#923
Replay (no store given)	(Vol, Aut)	refused #1022

*W missing; D only on unix. †Decodes as Aut (F) but neither D (relaxed commit) nor N (row-cap eviction) holds.

IV. Presence witness

A. Choice

W needs, at restore, a set $\mathrm{wit} \subseteq \mathcal{N}$ of names that were durably written, so that $\mathrm{wit} \setminus \mathrm{dom}\,\sigma'$ are the lost names. Fail-closed per stream (#910) is keyed by record name: $\mathrm{unavailable}(s) \iff \mathrm{name}(\mathrm{rec}(s)) \in \mathcal{U}$. A witness that does not name the lost record cannot feed $\mathcal{U}$. The candidates:

Witness	names loss	$O(1)$/new key	operator step
count $\lvert\mathrm{dom}\,\sigma\rvert$	no	yes	edit count
epoch per record	no	yes	edit epoch
manifest log	yes	yes	edit binary log
per-record marker	yes	yes	remove a file

A count or an epoch detects that something is missing but not which stream, so it can only fail the whole store closed: one lost file would close every stream. A manifest names the loss but turns the operator's per-stream recovery into editing a binary log, and its torn tail needs its own law. We choose the marker.

Definition 8 (Presence marker). For an authoritative native store, the marker of name n is an empty file $n.\mathsf{p}$ in the store's own directory. Let R be the set of record names listed, P the set of names with a marker, both from one listing.

Law 2 (Write order). An authoritative $\mathrm{put}(n, v)$ runs

$$\mathrm{write}(n.\mathtt{tmp});\ \mathrm{sync};\ \mathrm{rename}(n.\mathtt{tmp}, n);\ \mathrm{sync}_{\mathrm{dir}};$$
$$[\, n \notin P :\ \mathrm{create}(n.\mathsf{p});\ \mathrm{sync}_{\mathrm{dir}} \,]$$

and returns Ok only after the last step. An authoritative $\mathrm{rm}(n)$ runs $\mathrm{remove}(n.\mathsf{p}); \mathrm{sync}_{\mathrm{dir}}; \mathrm{remove}(n); \mathrm{sync}_{\mathrm{dir}}$.

Invariant 1 (Marker implies record). In every physical state the program and a crash reach, $P \subseteq R$.

Proof. A marker is created only after the rename that makes $n \in R$ is flushed, and removed before the record. A crash between any two steps of Law 2 leaves either P unchanged or the record already durable; by D the flushed prefix is what recovery sees. □

Law 3 (Restore). Restore classifies, from one listing:

$$\begin{aligned} n \in R,\ \text{decodes as its key} &\mapsto \mathsf{Filed}(k, v), \\ n \in R,\ \text{otherwise} &\mapsto \mathsf{Undecodable}(n), \\ n \in P \setminus R &\mapsto \mathsf{Missing}(n), \end{aligned}$$

and for $n \in R \setminus P$ creates n.p (flushed) before the store serves its first operation. The owner's $\mathcal{U}$ is $\{n : \mathsf{Undecodable}(n)\} \cup \{n : \mathsf{Missing}(n)\}$.

$R \setminus P$ is the crash between the rename and the marker: the record is whole and present, so it witnesses itself, and the repair restores $P = R$.

Proposition 1 (W holds). On a platform that discharges D, Laws 2 and 3 discharge Obligation 4 for every partial loss that leaves markers: a removed or renamed record file, a record lost by fsck, a rolled back rename of a new record.

Proof. Complete. Let $n \in \operatorname{dom} \sigma_k$; its first put completed, so $n \in P$ (Law 2). If n is lost and its marker is not, $n \in P \setminus R$, reported Missing. Sound. $\mathsf{Missing}(n)$ requires $n \in P$, and a marker is created only after a put of n flushed its rename (Law 2), so that record was durable. A crash before the marker's creation leaves $n \in R \setminus P$, never Missing: a put that never reached its rename is never reported lost. □

Proposition 2 (Total loss is undetectable). No witness stored in the failure domain of the records detects λ_{tot}.

Proof. After λ_{tot} the physical state equals that of a store never opened; any function of the physical state takes the same value on both. □

Total loss is therefore the documented reset: it is what the operator does by removing the replay store, and its consequences are those of clearing every stream (the recovery law of message::replay): a receiver may admit an unexpired transaction once more, and a sender restarts at sequence 0, which its destinations see as Stale or Fork until it passes their retained high watermark. A witness outside the domain (a sibling file in the data directory) would detect a vanished store directory, but the only producer of that state is the operator, whose clearing step it would then have to name; we do not add it.

Law 4 (Read agreement). An authoritative get(k) for name(k) $\in P$ whose record is not readable as the record of k returns Err, never $\mathsf{Ok}(\bot)$; $\mathsf{Ok}(\bot)$ means name(k) $\notin P$.

This closes the two cases added to #915 in review: a store whose root vanishes while the node runs answers Err for every witnessed name (as its persist already fails), and an upper-cased record file, which a case-insensitive volume still opens through get, is Missing to both get and scan because its lower-case marker remains.

Assumption 1 (Browsers lose whole origins). A browser deletes stored data per origin, never part of it [1]; this includes storage-pressure eviction of a non-persistent origin and WebKit's deletion of script-written data after seven days without user interaction.

Proposition 3 (IndexedDB needs no marker). Under Assumption 1 and N, an IndexedDB store discharges W with $P = \emptyset$.

Proof. Every IndexedDB write is a transaction, atomic by the specification [2]; with N the program removes no row it was not told to. By Assumption 1 the platform loses no proper subset of a database's rows, so no λ_{part} is reachable and W holds vacuously; total loss is Proposition 2. □

Cost. W adds one marker creation and one directory flush to the first put of each name, and nothing to later puts; restore lists one directory, as today. The replay store creates a name once per stream, so the cost is per new stream, not per transition.

Operator step. To clear one failed stream, stop the node and remove both <name> and <name>.p from transaction-replay. A store written before W has $P = \emptyset$ and is repaired by Law 3 at its first open: no migration is needed.

V. Survival per platform

D is a property of the platform's commit, so each platform discharges it separately.

Unix. Law 2 with sync = `File::sync_all`, which the Rust standard library maps to `fcntl(F_FULLFSYNC)` on Apple targets and to `fsync` elsewhere [3], and sync_{dir} the same on the directory. The store's own directory entry is flushed at open, with its ancestors. The rename-and-sync pattern is the one crash-consistency studies find portable across unix file systems [4]. test_durability model-checks the step plans over every crash point.

Windows. A directory cannot be opened for flushing through the standard library, so sync_{dir} is a no-op today and a completed rename can roll back (#915). The windows gateway is built and run in CI, so the target is in scope.

Law 5 (Write-through rename). On Windows every directory mutation of a Dur store is a rename by `MoveFileExW` with `MOVEFILE_REPLACE_EXISTING | MOVEFILE_WRITE_THROUGH`, which does not return until the move is on disk [5]: a put renames its flushed temporary file; a marker is created as a flushed temporary file renamed into place; a removal renames its file to .tmp and then deletes it.

A removal's deletion need not be durable: a .tmp left by a crash is removed at the next open (the index law of FileStorage), and it is not a record, so it never reappears.

IndexedDB. The specification makes the durability hint advisory [2]. What each engine does, read from its source:

Engine	default	strict
Chromium	relaxed since 121: complete fires once the data reaches the OS buffer [6]	flush before complete
Gecko	SQLite WAL, synchronous NORMAL (FULL under a pref) [7]	synchronous EXTRA
WebKit	SQLite WAL, synchronous left at the library default [8]	a FULL WAL checkpoint after commit

In WAL mode a commit under synchronous=NORMAL survives a process crash but may roll back after a system crash [9], which is Chromium's relaxed behaviour too. So default discharges D against a process crash in all three engines, and against a system crash in none that we can rely on.

Law 6 (Strict commit). Every read-write transaction of an IndexedDB store at $c = \mathsf{Dur}$ is opened with $\{\mathit{durability} : \text{strict}\}$, and the store reports Ok only on the transaction's complete event.

Under Law 6 D holds to the strength of each engine's strongest commit. We claim no more: whether that commit also flushes a drive's write cache is the engine's choice, as F_FULLFSYNC is ours on native Apple targets. rexie 0.4 opens no transaction with options; the binding is IdbDatabase::transaction_with_str_sequence_and_mode_and_options, which web-sys gates behind web_sys_unstable_apis, so the implementation either enables that cfg for the core crate or passes the options object through js-sys.

Corollary 1 (Authoritative reads need no write). IdbStorage restamps a row on every get to order eviction. Under N eviction never runs, so an authoritative get may be a read-only transaction.

VI. No eviction

Law 7 (Refuse, never evict). For $m \sqsupseteq \mathsf{Ret}$ on any backend: a put of a new name into a full store returns $\mathsf{Err}(\mathsf{BudgetExhausted})$ with the store unchanged; rewriting an existing name never needs room; an open with a capacity below the stored count fails.

Proposition 4 (Eviction is a forged absence). If an authoritative IndexedDB store evicts, W fails.

Proof. An evicted row $n \in \operatorname{dom}\sigma_k$ is gone with no marker (Proposition 3 has $P = \emptyset$), so restore reports nothing for it and its stream restarts from First: exactly #923. □

With Law 7 the browser replay store matches the native one (#914): a new stream past the bound fails closed, and the row capacity `TRANSACTION_REPLAY_STORE_MAX_RECORDS` becomes a refusal bound rather than a sizing hope. Whether a remote peer can drive a browser node to the cap stops mattering to safety; reaching it costs only new streams.

VII. Entry guards

Law 8 (Guards are a durable cache). The entry-guard store is opened at $(\mathsf{Dur}, \mathsf{Reb})$, DurableCache: writes discharge D (native: Law 2 without markers, Law 5; browser: Law 6); a guard record that is absent or does not decode is retired and the set is re-selected from scratch, logged at warn with the record's name.

Proposition 5 (Guards change only by reconciliation). Across a crash, the guard set the next selection reads is the last one written, unless a loss event (Definition 4) removed it.

Proof. D gives $\sigma'(n) \in \{\sigma_k(n), \sigma_{k+1}(n)\}$ for the one record per network; both are whole sets that reconcile_guard_state itself produced. □

The meaning stays Reb, which answers the policy question of #911: guards are privacy state whose loss is a downgrade (one re-selection of the first hop), not a safety failure, so failing onion routing closed on an undecodable record (F) would trade availability for nothing the guard set protects. Under D an undecodable guard record is no longer reachable by a crash, only by a loss event or a schema change, which the operator cleans before 1.0. The browser fallback to an in-memory guard store when IndexedDB cannot open is $(\mathsf{Vol}, \mathsf{Reb})$, admissible, and already logged.

VIII. Relay custody

A. Model

A relay element e carries a signed transaction of stream s (origin account, this node, class) at sequence q with digest h; write $e = (s, q, h)$. The actors:

- the carrier stored(d): the elements held for d at $d+1$, retired only by d's covering remove up to an element's dot;
- the receiver window $\mathcal{W}_s$, persisted at $(\mathsf{Dur}, \mathsf{Aut})$, with the pure transition observe$(\mathcal{W}_s, q, h) \in \{\mathsf{First}, \mathsf{Advance}, \mathsf{Late}, \mathsf{Replay}, \mathsf{Fork}, \mathsf{Stale}\}$;
- the application inbox, volatile; taking a message is where core's custody ends.

Today a drain admits h into $\mathcal{W}_s$ and queues the message; a restart before the take loses the queue but not $\mathcal{W}_s$, so the redelivery observes Replay and is retired (#936). The fact that is lost is "h was admitted for e and not taken".

Definition 9 (Custody set). The custody set of stream s is $\mathcal{K}_s \subseteq \mathbb{N} \times \text{Digest}$: the relay deliveries of s admitted and not yet taken. The receiver record of s is the pair $(\mathcal{W}_s, \mathcal{K}_s)$, one record under the existing key, so one write commits both atomically (A on both platforms).

Placing $\mathcal{K}_s$ in the receiver record, and not in the application or a separate store, is forced by two constraints: core never consults the application (the layering law), and a second record could not be committed atomically with $\mathcal{W}_s$, which would reopen the window between admission and its custody.

Law 9 (Custody). A drain pass first restricts custody to the carrier, $\mathcal{K}_s \leftarrow \mathcal{K}_s \cap \{(q,h) : (s,q,h) \in \text{stored}(d)\}$ (written only if it changed), then offers each element

$e = (s, q, h)$ it has not handed over in this process by the first arm that applies:

1) $(q, h) \in \mathcal{K}_s$: Requeue;
2) $|\bigcup_s \mathcal{K}_s| = \texttt{RELAY_INBOX_MAX_LEN}$: Defer;
3) observe$(\mathcal{W}_s, q, h) = (\mathcal{W}'_s, v)$ with v permitting dispatch: commit $(\mathcal{W}'_s, \mathcal{K}_s \cup \{(q, h)\})$, then queue;
4) otherwise: Retire,

where Requeue queues the message again without touching $\mathcal{W}_s$, and every queueing reserves room first (Defer when there is none, as today). When the application takes a message of e, core commits $(\mathcal{W}_s, \mathcal{K}_s \setminus \{(q, h)\})$, the release; the element is retired from the carrier only after its release committed. The live path is unchanged: it observes $\mathcal{W}_s$ alone.

Lemma 1 (Bounded custody). $|\bigcup_s \mathcal{K}_s| \leq$ `RELAY_INBOX_MAX_LEN` (64), so a receiver record grows by at most 64 entries of $8 + 32$ bytes over the whole store.

Proof. The Defer arm refuses a new entry at the bound, and the restriction step keeps only entries of elements still stored, of which a carrier holds at most 64. □

Proposition 6 (At least once across restart). Let the carrier satisfy (Dur, Ret) (#1021). In every history with any number of crashes, an element $e = (s, q, h)$ leaves the carrier only if (i) the application took a message of e, (ii) the live path would drop h too, or (iii) its retention bound elapsed.

Proof. The drain retires e in two arms. After a release: the release follows a take, case (i). In the last arm of Law 9: $(q, h) \notin \mathcal{K}_s$ and observe refuses. If observe is Replay, h is in $\mathcal{W}_s$, so it was admitted earlier either live (dispatched to the application, case (ii)) or by a relay delivery, which entered $\mathcal{K}_s$ in the same commit and left it only by a release (case (i)) or by the restriction step, which requires $e \notin \text{stored}(d)$. Fork and Stale are refusals of the live path, case (ii). A crash cannot separate the admission from its custody entry, nor a release from its effect, because each is one record write under D. Case (iii) is the retention law of #913. □

Proposition 7 (Duplicate window). The application receives a message of one element twice iff a crash falls after its take and before its release committed, and then at most once more per such crash.

Proof. Before the release, $(q, h) \in \mathcal{K}_s$ survives the crash and the redelivery is a Requeue. After it, $(q, h) \notin \mathcal{K}_s$ and $h \in \mathcal{W}_s$ or below its window, so the redelivery is refused and retired. □

The duplicate carries the same transaction id, so an application that needs exactly-once delivery deduplicates by it. Moving the release before the take returns would make the inbox's synchronous try_next perform a store write; we keep the take synchronous and accept the window, which is one record write long when the release is issued on the take's consumption signal.

B. Mail behind live traffic

#936 also asks that a relay message behind live traffic of the same origin be delivered. Inside the window it is: an empty slot gives Late. Below it, no receiver can.

Proposition 8 (No bounded receiver admits stale mail safely). Let $q < \text{rmin}(\mathcal{W}_s)$ and $(q, h) \notin \mathcal{K}_s$. No decision that depends only on bounded receiver state both admits every such e whose h was never dispatched and refuses every such e whose h was.

Proof. Take two histories that end in the same $(\mathcal{W}_s, \mathcal{K}_s)$: in the first, h arrived live, was dispatched, and at least `REORDER_WINDOW` later sequences followed; in the second, h was held, never delivered, and the same later sequences arrived live. A holder may hold a payload it forwarded (any hold inside the sender's proof lifetime is admissible), so e can be stored in both. The window retains only the last 32 digests, so the states agree, and any function of them decides both alike: admitting is a replay in the first, refusing is a loss in the second. Separating them needs every digest admitted within the sender's proof lifetime, which no bound on the store's size covers. □

So below the window the drain refuses as the live path does, which is the criterion #936 itself states (a refusal retires an element only when core would drop the same payload arriving live). Delivering such mail needs the sender to put it on a stream that live traffic does not advance, a protocol change outside this law.

IX. Implementation order

PR	Issue	Law
1	#923	Law 7 on IdbStorage; Cor. 1
2	#912	Law 6; SECURITY.md next to the native text
3	#915	Missing in ScannedRecord; Laws 2, 3, 4, 5
4	#911	DurableCache; Law 8
5	#936	Law 9; receiver record gains $\mathcal{K}_s$

The dependencies: 2 and 4 on the browser need the strict-commit seam of 2; 4 on Windows needs the write-through rename of 3; 5 needs nothing from 1–4, but its Proposition 6 holds in full only with #1021. PR 1 comes first because it is the only one that closes a safety gap reachable without a crash. #1021 and #1022 follow the batch.

Operator steps, in the CHANGELOG under Unreleased: PR 5 changes the receiver record, so an old record does not decode and fails its stream closed; the operator removes the transaction-replay store (native directory, browser database <storage name>/transaction-replay) while the node is stopped, accepting the reset of Proposition 2. PRs 1–4 need no step: an existing native store gains its markers at its first open, and the IndexedDB schema is unchanged.

X. Summary of laws

Predicate	Discharged by
presence, native	markers (Laws 2–4)
presence, browser	N and whole-origin loss (Prop. 3)
presence, total loss	undetectable; the documented reset
survival, unix	fsync, rename, directory fsync
survival, Windows	write-through rename (Law 5)
survival, browser	strict commit (Law 6)
presence of custody	$\mathcal{K}_s$ in the receiver record (Law 9)

References

[1] MDN Web Docs, "Storage quotas and eviction criteria," https://developer.mozilla.org/en-US/docs/Web/API/Storage_API/Storage_quotas_and_eviction_criteria, accessed 2026-10-23.

[2] W3C Web Applications Working Group, "Indexed database API 3.0," W3C Editor's Draft, https://w3c.github.io/IndexedDB/, section "Transaction durability"; accessed 2026-10-23.

[3] Apple, "fsync(2) and fcntl(2): F_FULLFSYNC," macOS manual pages, accessed 2026-10-23.

[4] T. S. Pillai, V. Chidambaram, R. Alagappan, S. Al-Kiswany, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "All file systems are not created equal: On the complexity of crafting crash-consistent applications," in 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI '14). USENIX Association, 2014, pp. 433–448.

[5] Microsoft, "MoveFileExW function (winbase.h): MOVEFILE_WRITE_THROUGH," https://learn.microsoft.com/en-us/windows/win32/api/winbase/nf-winbase-movefileexw, accessed 2026-10-23.

[6] Chrome for Developers, "A change to the default durability mode in IndexedDB," https://developer.chrome.com/blog/indexeddb-durability-mode-now-defaults-to-relaxed, default changes from strict to relaxed in Chrome 121; accessed 2026-10-23.

[7] Mozilla, "IndexedDB parent actors (ActorsParent.cpp)," https://github.com/mozilla-firefox/firefox/blob/main/dom/indexedDB/ActorsParent.cpp, BeginWriteTransaction and GetDefaultSynchronousMode; accessed 2026-10-23.

[8] WebKit, "IndexedDB SQLite backing store (SQLiteIDBBackingStore.cpp)," https://github.com/WebKit/WebKit/blob/main/Source/WebCore/Modules/indexeddb/server/SQLiteIDBBackingStore.cpp, commitTransaction; accessed 2026-10-23.

[9] SQLite, "Write-ahead logging, and PRAGMA synchronous," https://www.sqlite.org/wal.html, https://www.sqlite.org/pragma.html, accessed 2026-10-23.